

RentalSathi

Rental Property Management System

Full-Stack Web Application

A Project Report

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Computer Applications / Bachelor of Science in Computer
Science

Submitted by:

Ishika Khadka

GitHub Repository:

`github.com/ishikakhadka/RentalSathi_fullstack`

Date of Submission: March 2026

Acknowledgement

I would like to express my sincere gratitude to all those who contributed — directly or indirectly — to the successful completion of the RentalSathi project.

First and foremost, I extend my heartfelt thanks to my project supervisor and faculty members for their continuous guidance, constructive feedback, and encouragement throughout the development process. Their expertise in software engineering and web development helped me navigate complex technical challenges and maintain focus on the project objectives.

I am grateful to my institution for providing the necessary resources, infrastructure, and a conducive learning environment that made this project possible.

I would also like to thank the open-source community behind the technologies used in this project — particularly the contributors to Node.js, Express.js, Next.js, MongoDB, and the broader JavaScript ecosystem. Their well-documented libraries and frameworks were instrumental in the development of this system.

Special thanks to my classmates and peers for their valuable suggestions during testing and review sessions, and for helping identify usability issues that improved the final product significantly.

Finally, I am deeply grateful to my family for their unwavering moral support, patience, and encouragement during the entire duration of this project.

Ishika Khadka

March 2026

Abstract

RentalSathi is a full-stack web application designed to streamline the rental property management process by connecting landlords and tenants through a unified digital platform. The name “Sathi” means companion in Nepali, reflecting the system’s goal of being a reliable companion in the property rental journey.

The platform addresses the inefficiencies of traditional, offline-based property search methods that are common in Nepal – particularly the reliance on word-of-mouth, physical visits, and newspaper advertisements. RentalSathi provides a centralised system where landlords can list, manage, and update their rental properties, while tenants can browse, search, filter by category, view property details, save properties to a personal basket, and communicate with landlords in real time.

The system is developed using a modern technology stack: the frontend is built with Next.js 15 (React 19, TypeScript, Tailwind CSS, Material UI) and the backend is a Node.js/Express.js REST API. Data is persisted in MongoDB using the Mongoose ODM. Authentication is handled via JWT (JSON Web Tokens) with role-based access control distinguishing between landlord and tenant roles. Real-time communication is facilitated through Socket.IO. Input validation is enforced on both the client and server using Yup schema validation.

Key features include user registration and login, property listing with pagination, category-based filtering, a property basket (cart) for tenants, CRUD operations for landlords, and a live chat module. The system is deployed with the frontend on Vercel and the backend on a cloud server, making it publicly accessible.

This report documents the full software development lifecycle of RentalSathi – from problem identification and requirements analysis to system design, implementation, testing, and future recommendations.

Keywords: Rental management, full-stack web application, Next.js, Node.js, MongoDB, JWT, real-time communication, REST API, role-based access control.

Contents

Acknowledgement	i
Abstract	i
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitation	3
1.5 Development Methodology	4
1.6 Report Organization	4

2	Background Study and Literature Review	5
2.1	Background Study	5
2.2	Literature Review	6
3	System Analysis	7
3.1	System Overview and Data Flow	7
3.1.1	Data Flow Diagrams	7
3.2	Requirement Analysis	10
3.2.1	Functional Requirements	10
3.2.2	Non-Functional Requirements	10
3.3	Feasibility Analysis	10
3.3.1	Technical Feasibility	10
3.3.2	Schedule Feasibility	12
3.3.3	Operational Feasibility	13
3.4	Analysis	13
3.4.1	Refined Class Diagram	13
3.4.2	Activity Diagram	14
4	System Design	19
4.1	Design	19
4.1.1	System Architecture	19
4.1.2	Component Diagram	19
4.1.3	Algorithm Details	21
4.1.4	System Workflows	22
5	Implementation and Testing	27
5.1	Implementation	27
5.1.1	Tools Used	27
5.1.2	User Registration and Login Workflow	28
5.1.3	Property Management Workflow	30
5.1.4	Tenant Property Browsing and Details	30
5.1.5	Property Basket Management	30
5.1.6	Property Deletion and Advanced Operations	30
5.2	Testing	33
5.2.1	Test Cases for Unit Testing	33
5.2.2	Test Cases for System Testing	36
5.3	Result Analysis	37
6	Conclusion and Future Recommendations	38

List of Figures

3.1	DFD Level 0 – Context Diagram: System Scope and External Entities . .	8
3.2	DFD Level 1 – System Decomposition: Major Processes and Data Flows	8
3.3	DFD Level 2 – User Authentication Process: Registration and Login Logic	9
3.4	Entity-Relationship Diagram (ERD): RentalSathi MongoDB Collections .	15
3.5	User Collection Schema: User Collection Schema Class Diagram	16
3.6	Property Collection Schema: Property Collection Schema Class Diagram	16
3.7	Cart Collection Schema: Cart Collection Schema Class Diagram	17
3.8	Activity Diagram – Tenant: Add Property to Basket: Tenant adds a prop- erty to the basket through the middleware chain	18
4.1	Three-Tier System Architecture: RentalSathi Three-Tier System Archi- tecture	20
4.2	Route Flow Architecture: Client-side routing through Next.js App Router based on role and auth status	23
4.3	Authentication Flow: Complete authentication flow from login through token-based request handling	24
4.4	Property Listing Flow: Property listing workflow including filtering, pagi- nation, and data retrieval	25
4.5	Tenant Browse and Cart Flow: Tenant browsing and basket management workflow	26
4.6	Backend Request Lifecycle: Complete backend request lifecycle from Ex- press.js through middleware to response	26
4.7	Real-Time Chat Flow: Socket.IO-based real-time chat architecture be- tween chat popup and server	29
5.1	User Registration Sequence: Interaction between client, frontend, Express API, and MongoDB during registration	30
5.2	User Login Sequence: Credential verification and JWT token issuance upon successful authentication	31
5.3	Add Property Sequence: Landlord creates a new property listing through the middleware chain	32
5.4	Tenant Browse Properties Sequence: Paginated property listing retrieval via React Query and MongoDB aggregation	32

5.5	View Property Details Sequence: Full property detail retrieval by ID with authentication and validation	33
5.6	Add to Property Basket Sequence: Tenant saves a property to basket with duplicate and existence checks	35
5.7	Delete Property Sequence: Landlord deletes an owned property with ownership verification middleware	35

List of Tables

3.1	Functional Requirements – User Module	10
3.2	Functional Requirements – Property Module	11
3.3	Functional Requirements – Cart Module	11
3.4	Functional Requirements – Chat Module	11
3.5	Non-Functional Requirements	12
3.6	Development Schedule	13
3.7	User Class Attributes	14
3.8	Property Class Attributes	14
3.9	Cart Class Attributes	17
5.1	Tools and Technologies Used	27
5.2	Unit Test Cases	33
5.3	System Test Cases	36

Chapter 1

Introduction

1.1 Introduction

The housing and rental market is one of the most dynamic sectors in any economy. In Nepal and many developing countries, the process of finding a rental property remains largely informal, fragmented, and inefficient. Tenants rely heavily on word-of-mouth recommendations, physical scouting of neighbourhoods, and scattered online listings spread across social media platforms with little standardisation or trustworthiness.

RentalSathi was conceived to address this gap. It is a web-based rental property management platform that brings structure, transparency, and convenience to the rental property search and listing process. By providing dedicated workflows for both landlords and tenants, RentalSathi eliminates intermediaries, reduces time spent on property searches, and creates a trustworthy environment for both parties.

The system provides landlords with tools to list, edit, and manage their properties digitally, with categorisation and image support. Tenants are given a rich browsing experience with pagination, category filtering, detailed property views, a “Property Basket” for saving shortlisted properties, and a real-time chat feature for direct communication with landlords.

This chapter presents an overview of the problem being solved, the objectives set for the system, its scope and limitations, the development methodology followed, and an outline of the remaining chapters of this report.

1.2 Problem Statement

Despite the growing demand for rental properties in urban areas of Nepal, the process of searching for and listing rental properties remains predominantly unstructured and offline. The key problems that exist in the current scenario include:

1. **Lack of a centralised platform:** Property listings are scattered across physical notices, social media posts, and informal networks, making it difficult for tenants to compare multiple options efficiently.

2. **Information asymmetry:** Tenants often have limited information about properties before physically visiting them, leading to wasted time and resources.
3. **No role-based access:** Existing general-purpose platforms do not differentiate the needs and workflows of landlords versus tenants, resulting in poor user experience for both.
4. **Absence of real-time communication:** Communication between landlords and tenants typically happens through phone calls or in-person meetings, with no integrated messaging facility.
5. **No shortlisting mechanism:** Tenants have no structured way to shortlist and compare multiple properties of interest in one place.
6. **Security and trust:** Unverified listings on informal platforms create opportunities for fraud and misinformation.

These challenges collectively result in an inefficient rental market experience for all stakeholders. RentalSathi is designed to directly address each of these issues through a feature-rich, role-aware digital platform.

1.3 Objectives

The primary objective of this project is to design and implement a full-stack rental property management system that connects landlords and tenants through a secure, user-friendly web platform. The specific objectives are:

1. To develop a **role-based authentication system** that distinctly manages landlord and tenant accounts using JWT-based secure authentication.
2. To provide landlords with the ability to **create, read, update, and delete (CRUD)** property listings with rich details including title, location, price, number of rooms, category, description, and images.
3. To provide tenants with a **paginated, filterable property browsing experience** with full property detail views.
4. To implement a **Property Basket** feature allowing tenants to save, view, and manage shortlisted properties.
5. To integrate a **real-time chat module** using Socket.IO to facilitate direct communication between tenants and landlords.

6. To enforce **robust input validation** on both the client side (Yup + Formik) and the server side (Yup middleware) for high data integrity.
7. To build the system using **modern, industry-standard technologies** (Next.js 15, Node.js, Express.js, MongoDB, React Query) that ensure scalability and maintainability.
8. To **deploy the application publicly** with the frontend on Vercel and the backend on a cloud hosting service.

1.4 Scope and Limitation

Scope

The RentalSathi system encompasses the following scope:

- User registration and login for two distinct roles: landlord and tenant.
- Full CRUD management of property listings by landlords, including category-based classification (Villa, Apartment, Commercial, Homestay, Flats, Warehouse).
- Paginated property listing and category-based browsing for tenants.
- Detailed property view for any authenticated user.
- Property Basket (cart) management for tenants – add, remove, flush, list, and count saved properties.
- Real-time chat popup between authenticated users via Socket.IO.
- RESTful API backend with JWT authentication and Yup validation middleware.
- Deployment on publicly accessible cloud infrastructure.
- Responsive frontend interface compatible with modern web browsers.

Limitations

- **No payment integration:** The system does not support online rent payments or deposit transactions.
- **No advanced search:** Full-text search and map-based location search are not implemented in this version.
- **No image upload service:** Property images are currently stored as URL strings; cloud-based image upload (e.g., Cloudinary) is not integrated.

- **No email verification:** User registration does not require email confirmation.
- **No admin panel:** There is no administrative interface for platform-level management (e.g., user banning, listing moderation).
- **Basic chat:** The chat module is rudimentary and does not persist chat history in the database.
- **No mobile application:** The platform is limited to web browsers; native iOS/Android applications are outside the current scope.

1.5 Development Methodology

RentalSathi was developed following an **Agile-inspired iterative development** approach. Rather than a rigid, phase-locked waterfall model, development proceeded through iterative cycles where features were designed, built, tested, and refined progressively.

Key aspects of the methodology:

- **Feature-driven development:** The system was broken down into independent functional modules – user authentication, property management, cart, and chat – and each was developed end-to-end before integration.
- **Continuous integration:** Changes were committed regularly to the Git repository, enabling version control and rollback capability.
- **Test-then-fix:** After each feature was implemented, manual and unit testing was performed to identify and fix defects before moving to the next feature.
- **Separation of concerns:** Frontend (Next.js) and backend (Express.js) were developed independently with a well-defined REST API contract, enabling parallel development cycles.
- **Incremental deployment:** The frontend was deployed to Vercel at each major milestone, allowing real-world testing against the live backend API.

The development cycle for each module followed the pattern: Requirements definition → Database schema design → API route implementation → Frontend component development → Integration testing → Review and refinement.

1.6 Report Organization

The remainder of this report is organized as follows:

Chapter 2 – Background Study and Literature Review: Reviews existing rental platforms and related technologies. Discusses findings from literature that informed design decisions.

Chapter 3 – System Analysis: Presents functional and non-functional requirements, feasibility analysis across technical, schedule, and operational dimensions, along with class and activity diagrams.

Chapter 4 – System Design: Details the architectural and component design, including the component diagram and key algorithm descriptions.

Chapter 5 – Implementation and Testing: Describes the tools and technologies used, the implementation details, test cases for unit and system testing, and result analysis.

Chapter 6 – Conclusion and Future Recommendations: Summarises the project outcome and proposes directions for future development.

Chapter 2

Background Study and Literature Review

2.1 Background Study

The concept of online real estate and rental platforms has evolved significantly since the early 2000s. Globally, platforms such as Zillow, Airbnb, Rightmove, and OLX have transformed how people search for properties. These platforms demonstrate that digitalising the property rental workflow improves reach, transparency, and efficiency for all stakeholders.

In the context of Nepal, the rental market has been growing rapidly due to urbanisation, particularly in cities like Kathmandu, Pokhara, and Biratnagar. However, the adoption of technology in this sector remains limited. Most property searches still happen through local brokers (dalals), physical notice boards, or informal Facebook groups.

A few portals exist (e.g., Hamrobazar, Nepaliaawas) but their interfaces are outdated, lack real-time features, and do not provide structured role management for landlords and tenants.

Technology background:

- **MERN-adjacent stack:** The combination of MongoDB, Express.js, React (via Next.js), and Node.js has become the dominant choice for full-stack JavaScript web applications due to its unified language (JavaScript/TypeScript) across all layers.
- **Next.js App Router:** Next.js 13+ introduced the App Router, which supports React Server Components, nested layouts, and improved routing – features leveraged in RentalSathi’s route structure.
- **JWT authentication:** Stateless token-based authentication with JWTs is the industry standard for modern REST APIs, avoiding the overhead of server-side session management.
- **Socket.IO:** A widely used library that enables real-time, bidirectional event-based communication between web clients and servers using WebSocket protocol with automatic fallback.
- **React Query (TanStack Query):** A data-fetching and server state management library for React that handles caching, background refetching, and loading states, significantly simplifying API integration in Next.js.

2.2 Literature Review

Several studies and technical reports have examined online rental management systems and related technologies. The following review highlights the most relevant findings:

[1] **Wafula et al. (2022) – “Design and Development of a Web-Based House Rental System”:** This paper examined a house rental management system for African urban markets and found that role-based access control (RBAC) and responsive design were among the top usability requirements cited by users. Their findings informed RentalSathi’s strict role separation (landlord vs. tenant) enforced at the API middleware level.

[2] **Kumar & Singh (2021) – “A Comparative Study of NoSQL vs. SQL Databases for Real Estate Applications”:** This study found that NoSQL databases, particularly MongoDB, are better suited for property management applications due to the varying and extensible nature of property attributes. The schema-flexible document model of MongoDB accommodated RentalSathi’s property schema (which includes variable-length descriptions, optional images, and multiple categories) more naturally than a relational model.

[3] Verma et al. (2020) – “Real-Time Communication in Web Applications: A Study of WebSocket and Socket.IO”: This paper demonstrated that Socket.IO’s fallback mechanisms (long polling → WebSocket upgrade) make it more reliable than raw WebSocket for deployment in environments with variable network conditions, supporting its choice for RentalSathi’s chat feature.

[4] TanStack Query Documentation (2024) – “Server State Management in React”: The official documentation and community patterns for React Query provided the architectural basis for RentalSathi’s frontend data-fetching strategy, particularly the use of `useQuery` for property lists and `useMutation` for cart operations.

[5] OWASP (2023) – “JSON Web Token Best Practices”: The OWASP guide on JWT security identified key risks such as algorithm confusion attacks and insecure secret storage. RentalSathi’s implementation stores the JWT secret in environment variables and enforces server-side role verification on every protected route, in line with OWASP recommendations.

Chapter 3

System Analysis

3.1 System Overview and Data Flow

This section provides visual representations of the RentalSathi system’s architecture and key data flows.

3.1.1 Data Flow Diagrams

Figure 3.1 shows the Level 0 Context Diagram, identifying all external actors (Guest, Landlord, Tenant) and their interactions with the central platform.

Figure 3.2 decomposes the system into five major functional processes: user registration/login, property management, property browsing, cart management, and real-time chat.

Figure 3.3 details the authentication subprocess, showing the decision paths for both user registration and login including validation, hashing, and token generation.

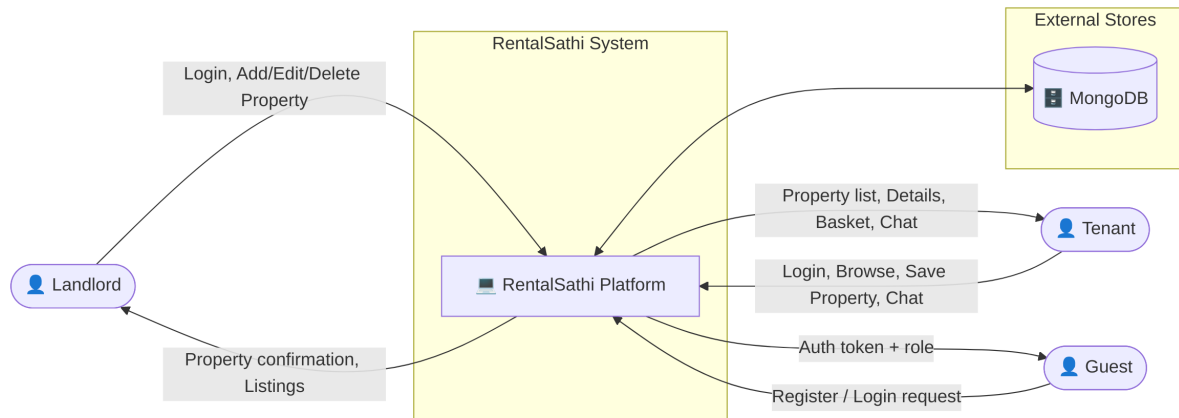


Figure 3.1: DFD Level 0 – Context Diagram: System Scope and External Entities

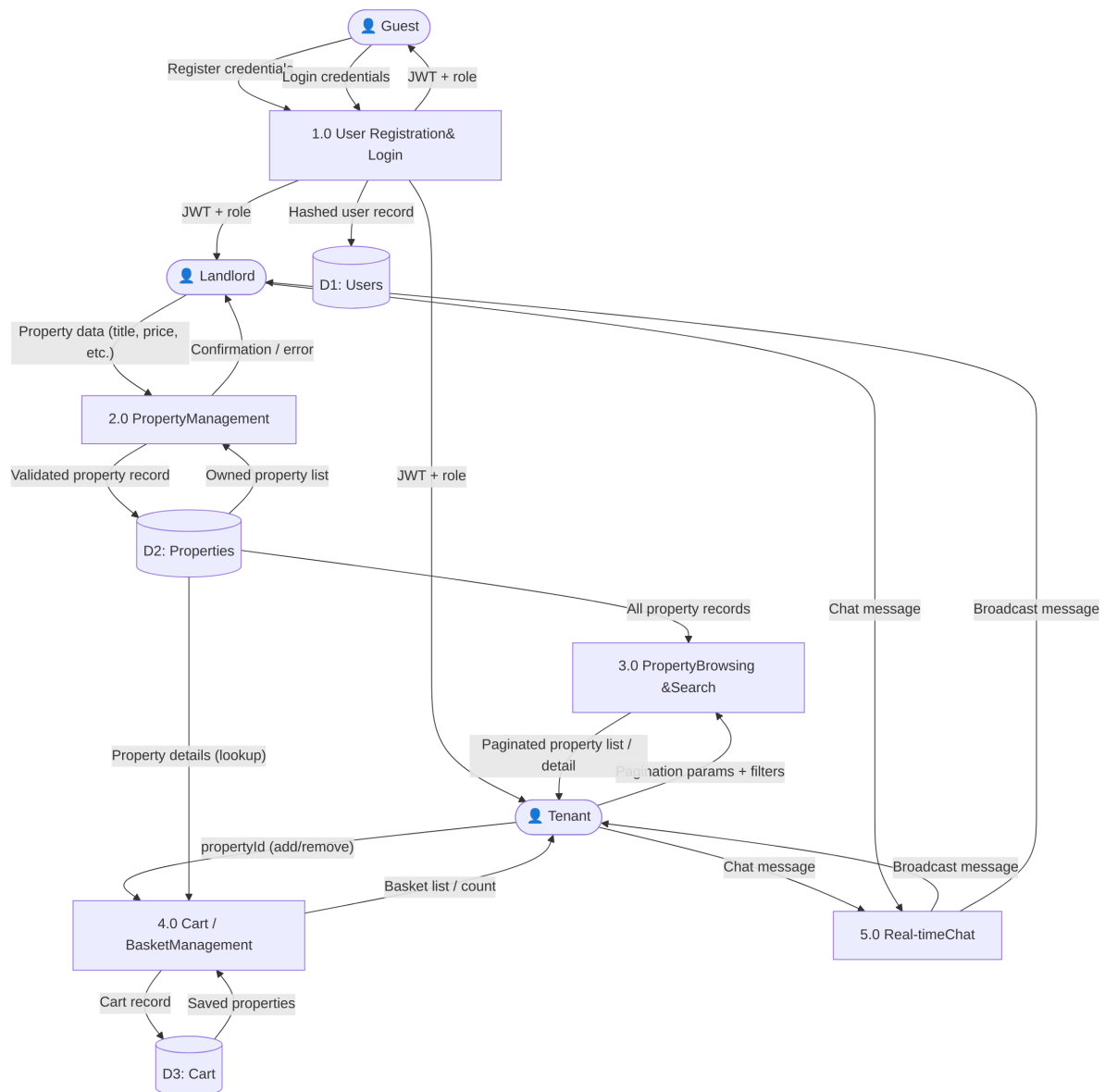


Figure 3.2: DFD Level 1 – System Decomposition: Major Processes and Data Flows

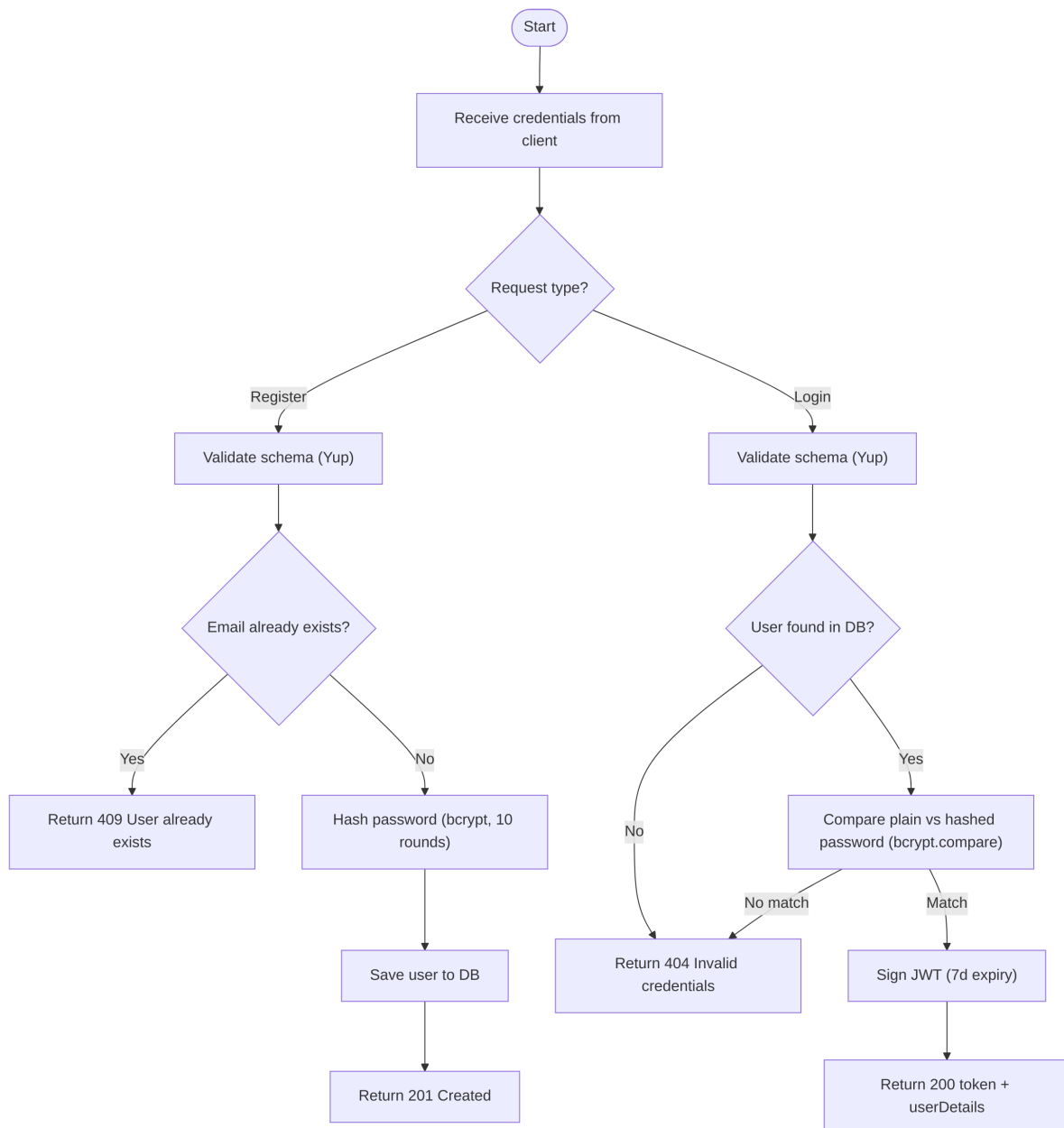


Figure 3.3: DFD Level 2 – User Authentication Process: Registration and Login Logic

3.2 Requirement Analysis

3.2.1 Functional Requirements

Functional requirements define the specific behaviours and functions the system must perform. The functional requirements for RentalSathi are listed below by module:

User Module

Table 3.1: Functional Requirements – User Module

ID	Feature	Description
FR-U1	User Registration	The system shall allow a new user to register with first name, last name, email, password, gender, date of birth, role (landlord or tenant), and address.
FR-U2	Unique Email	The system shall reject registration if the provided email already exists in the database.
FR-U3	Password Hashing	The system shall hash all passwords using bcrypt (salt rounds = 10) before storing them.
FR-U4	User Login	The system shall authenticate a user by verifying their email and hashed password and issue a signed JWT on success.
FR-U5	Token Expiry	The system shall issue JWTs with a 7-day expiry period.
FR-U6	Role Persistence	The system shall persist the user's role in localStorage upon login for client-side guard enforcement.

Property Module

Cart (Property Basket) Module

Chat Module

3.2.2 Non-Functional Requirements

3.3 Feasibility Analysis

3.3.1 Technical Feasibility

The technical feasibility of RentalSathi is assessed based on the availability and maturity of the technologies chosen.

- **Next.js 15 / React 19:** Mature, heavily used frameworks with extensive documentation, community support, and production-ready deployments on Vercel.

Table 3.2: Functional Requirements – Property Module

ID	Feature	Description
FR-P1	Add Property	An authenticated landlord shall be able to add a property with title, location, price, number of rooms, category, description, and an optional image URL.
FR-P2	Edit Property	An authenticated landlord shall be able to update details of a property they own.
FR-P3	Delete Property	An authenticated landlord shall be able to delete a property they own.
FR-P4	List (Landlord)	The system shall return a paginated list of properties owned by the logged-in landlord, sorted by most recent.
FR-P5	List (Tenant)	The system shall return a paginated list of all active properties visible to tenants, with summary fields.
FR-P6	Property Detail	Any authenticated user shall be able to view the full detail of a property by its ID.
FR-P7	Category Filter	Tenants shall be able to filter properties by category.
FR-P8	Ownership Guard	The system shall prevent a landlord from editing or deleting properties they do not own.

Table 3.3: Functional Requirements – Cart Module

ID	Feature	Description
FR-C1	Add to Basket	An authenticated tenant shall be able to save a property to their basket.
FR-C2	No Duplicates	The system shall prevent the same property from being added to the basket twice.
FR-C3	Remove from Basket	An authenticated tenant shall be able to remove a specific item from their basket.
FR-C4	Flush Basket	An authenticated tenant shall be able to clear their entire basket in one operation.
FR-C5	List Basket	The system shall return all saved properties for the logged-in tenant with full property details via lookup.
FR-C6	Basket Count	The system shall return the count of items in the tenant's basket for display in the navigation badge.

Table 3.4: Functional Requirements – Chat Module

ID	Feature	Description
FR-Ch1	Real-Time Chat	The system shall allow authenticated users to send and receive messages in real time via Socket.IO.
FR-Ch2	Chat Popup	The frontend shall provide a collapsible chat popup component accessible from property detail pages.

- **Node.js / Express.js 5:** A well-established server-side JavaScript runtime and minimal web framework, widely used for REST API development.

Table 3.5: Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	API responses for paginated property lists shall be returned within 2 seconds under normal load.
NFR-2	Security	All passwords shall be hashed using bcrypt. JWTs shall be signed with a secret key stored in environment variables.
NFR-3	Security	All protected API routes shall validate the JWT token and user role before processing the request.
NFR-4	Usability	The interface shall be responsive and usable on screens with a minimum width of 360px.
NFR-5	Maintainability	The codebase shall follow a modular structure separating controllers, models, services, and validation.
NFR-6	Scalability	The MongoDB schema and pagination design shall support growth to thousands of property listings without redesign.
NFR-7	Availability	The deployed system shall target 99% uptime leveraging Vercel's global CDN for the frontend.
NFR-8	Validation	All user inputs shall be validated using Yup schemas on both the client and server.
NFR-9	Portability	The system shall run on any modern web browser (Chrome, Firefox, Edge, Safari) without plugins.

- **MongoDB / Mongoose 8:** MongoDB Atlas provides a free-tier cloud database suitable for development and small-scale production. Mongoose provides a stable ODM layer.
- **Socket.IO 4:** A reliable library for real-time WebSocket communication with broad browser support.
- **JWT / bcrypt:** Standard libraries for authentication and password hashing with well-understood security properties.

All selected technologies are open-source, freely available, and require no specialized hardware. The system can be developed on a standard developer laptop. **Conclusion:** Technically feasible.

3.3.2 Schedule Feasibility

The project was scoped for a single academic semester. The major milestones were:

The project was completed within the planned timeline. **Conclusion:** Schedule feasible.

Table 3.6: Development Schedule

Phase	Activities	Duration
Phase 1	Requirements gathering, tech stack selection, DB design	1 week
Phase 2	Backend: User module (auth, register, login)	1 week
Phase 3	Backend: Property module (CRUD, pagination)	1 week
Phase 4	Backend: Cart module and middleware	1 week
Phase 5	Frontend: Auth pages (login, register, guards)	1 week
Phase 6	Frontend: Landlord dashboard and property forms	1 week
Phase 7	Frontend: Tenant dashboard, browsing, basket	1 week
Phase 8	Socket.IO chat, integration testing, deployment	1 week
Phase 9	Documentation and report writing	1 week

3.3.3 Operational Feasibility

- **End-user accessibility:** The web-based nature of the application means users require only a modern web browser and an internet connection – no installation required.
- **Low operational cost:** Vercel’s free tier covers the frontend. MongoDB Atlas free tier covers the database. Backend hosting on platforms like Render provides a free-tier starter plan.
- **User training:** The interface is intuitive and follows standard web UX patterns. Minimal training is required for both landlords and tenants.
- **Maintenance:** The modular codebase structure allows individual modules to be updated independently without affecting the rest of the system.

Conclusion: Operationally feasible.

3.4 Analysis

3.4.1 Refined Class Diagram

The logical data model of RentalSathi maps directly to three Mongoose models with the following relationships:

- **User (1) → (many) Property:** A landlord can own multiple properties.
- **User (1) → (many) Cart:** A tenant can have multiple saved properties.
- **Property (1) → (many) Cart:** A property can appear in multiple tenants’ baskets.

Figure 3.4 shows the Entity-Relationship Diagram (ERD) for the three collections showing all relationships and data flow between entities.

Figure 3.5 illustrates the User collection schema with all attributes, types, and constraints.

Figure 3.6 shows the Property collection schema including the landlord foreign key relationship.

Figure 3.7 depicts the Cart collection schema with both foreign keys to User (tenantId) and Property (propertyId).

The detailed attribute tables for each collection follow.

Table 3.7: User Class Attributes

Attribute	Type	Constraints
_id	ObjectId	Primary key (auto)
email	String	Required, unique, max 100
password	String	Required, bcrypt hashed
firstName	String	Required, max 100
lastName	String	Required, max 100
dob	Date	Optional
gender	String	Enum: male, female, other
role	String	Enum: tenant, landlord
address	String	Required, max 255

Table 3.8: Property Class Attributes

Attribute	Type	Constraints
_id	ObjectId	Primary key (auto)
title	String	Required, max 255
location	String	Required, max 255
price	Number	Required, min 0
noOfRooms	Number	Required, min 1
category	String	Enum: Villa, Apartment, etc.
image	String	Optional, nullable
description	String	Required, min 10, max 1000
landlordId	ObjectId	FK → User
createdAt	Date	Auto (timestamps)
updatedAt	Date	Auto (timestamps)

3.4.2 Activity Diagram

Figure 3.8 shows the activity flow for Tenant Adding a Property to the Basket, illustrating the full middleware chain and decision points.

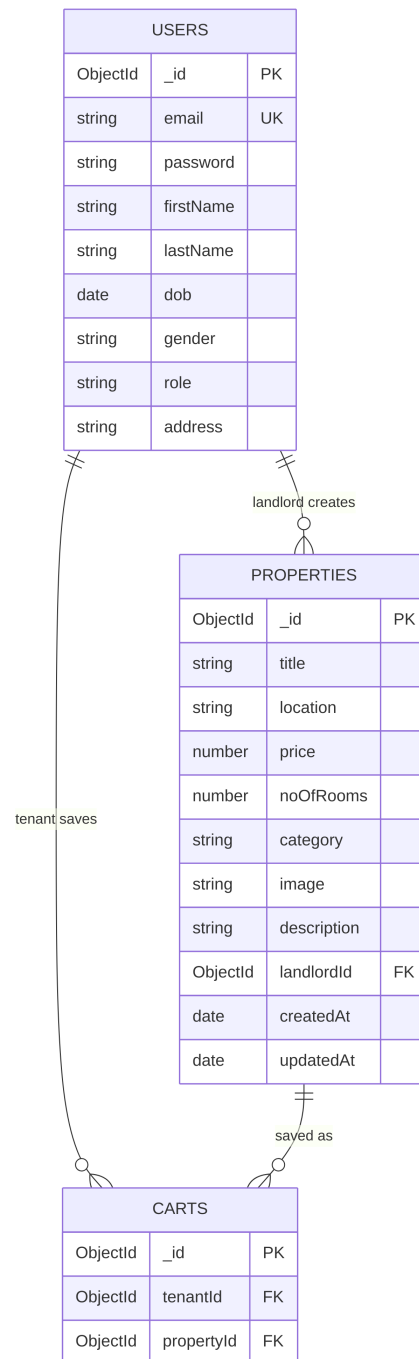


Figure 3.4: Entity-Relationship Diagram (ERD): RentalSathi MongoDB Collections

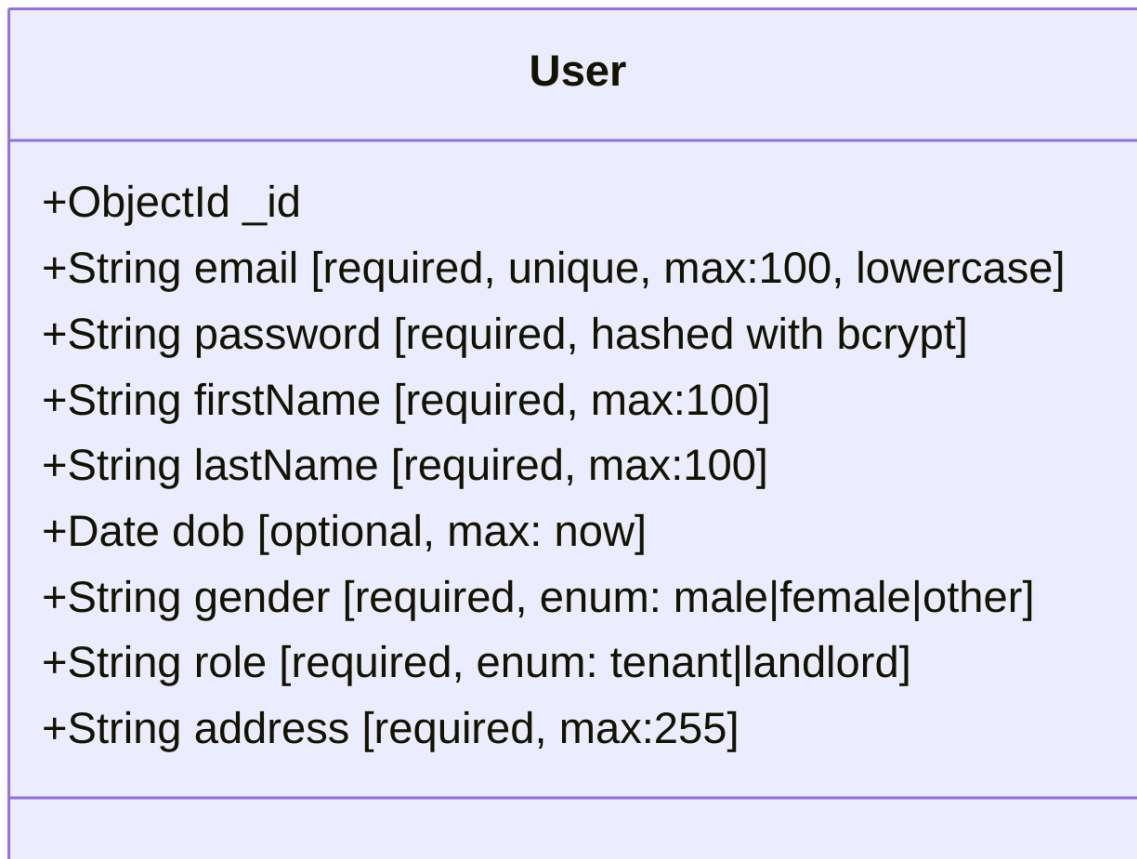


Figure 3.5: User Collection Schema: User Collection Schema Class Diagram

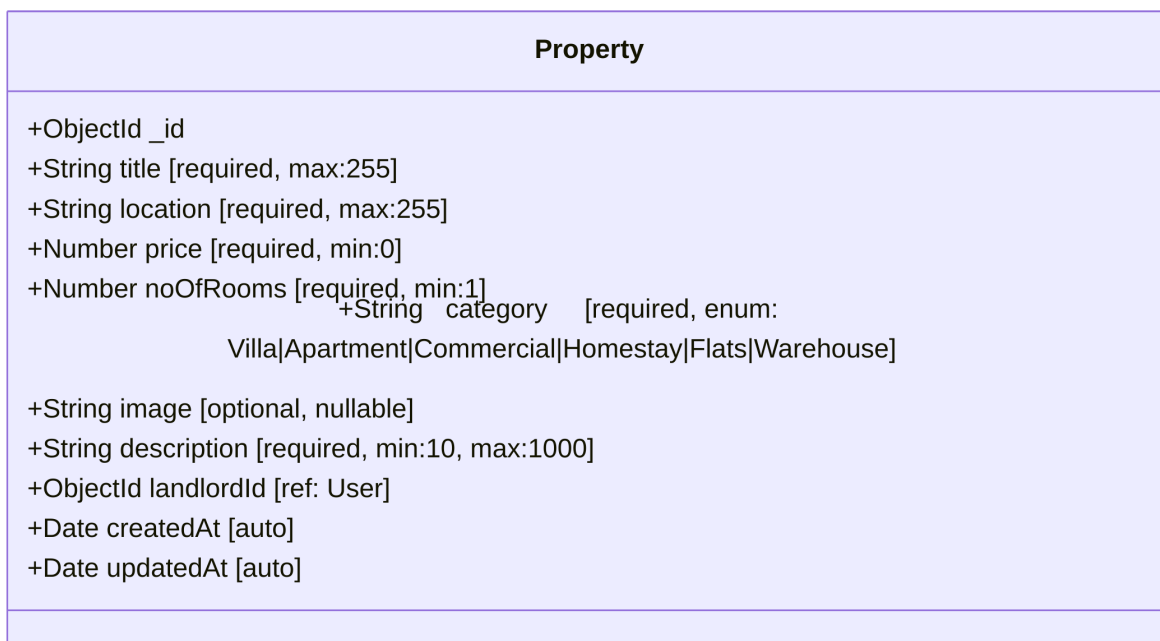


Figure 3.6: Property Collection Schema: Property Collection Schema Class Diagram

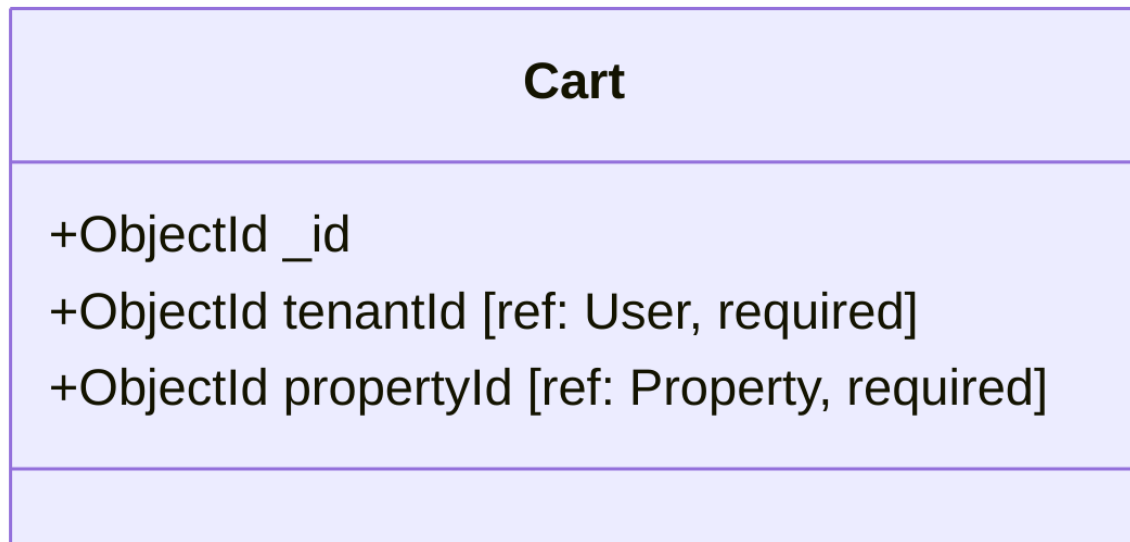


Figure 3.7: Cart Collection Schema: Cart Collection Schema Class Diagram

Table 3.9: Cart Class Attributes

Attribute	Type	Constraints
_id	ObjectId	Primary key (auto)
tenantId	ObjectId	FK → User (tenant)
propertyId	ObjectId	FK → Property

A similar activity flow applies to landlord property management (add → validate → check ownership → persist → confirm).

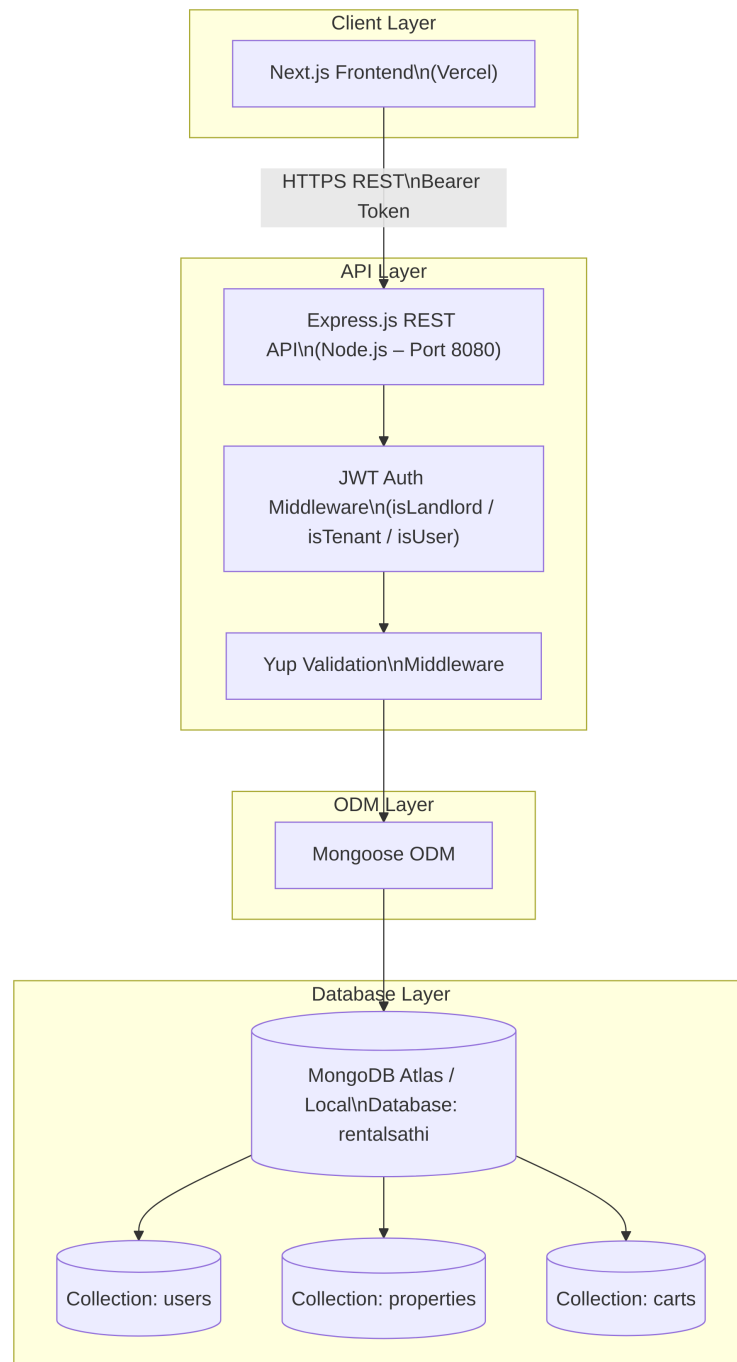


Figure 3.8: Activity Diagram – Tenant: Add Property to Basket: Tenant adds a property to the basket through the middleware chain

Chapter 4

System Design

4.1 Design

4.1.1 System Architecture

Figure 4.1 illustrates the three-tier architecture of RentalSathi showing the interaction between the client, API, and database layers.

4.1.2 Component Diagram

The RentalSathi system is composed of the following major components and their interactions:

Frontend Components (Next.js – App Router):

- **(guest) Layout:** Renders public pages (Login, Register) wrapped in GuestGuard.
- **(auth) Layout:** Wraps all authenticated routes. Contains role-specific sub-routes for landlord and tenant.
- **LandlordGuard / TenantGuard:** Client-side guards that check localStorage for a valid token and role. Redirect unauthenticated users to /login.
- **Navbar / NavbarLandlord:** Role-specific navigation bars.
- **AddPropertyForm / EditPropertyForm:** Formik + Yup powered forms for property CRUD.
- **PropertyCard / TenantContainer:** Card-based display components for property listings.
- **PropertyDetails:** Full-detail view component with basket and chat action buttons.
- **PropertyBasketContainer:** Displays saved properties with remove functionality.
- **ChatPopup:** Socket.IO-based floating chat component.

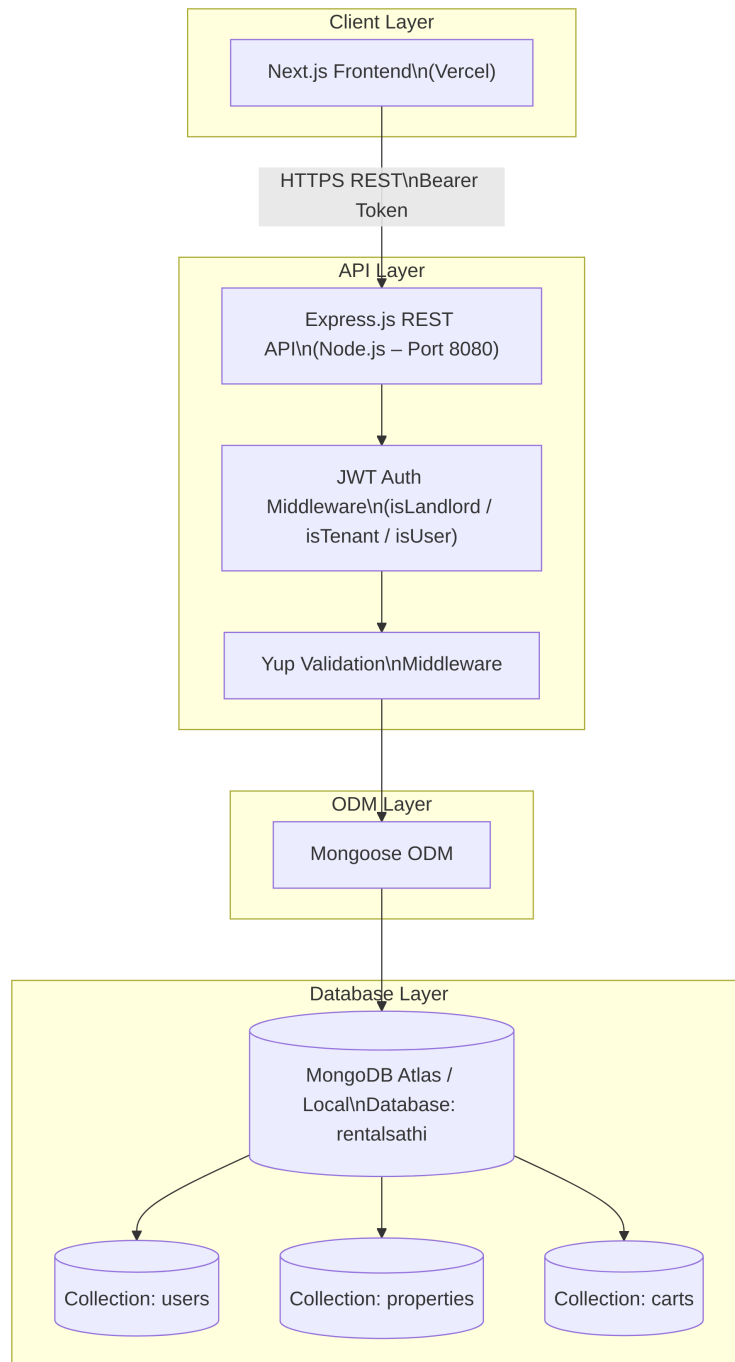


Figure 4.1: Three-Tier System Architecture: RentalSathi Three-Tier System Architecture

- **ReactQueryClientProvider:** Wraps the application with TanStack Query's QueryClient.
- **axios.instance.ts:** Pre-configured Axios instance that automatically attaches the `Authorization: Bearer <token>` header from `localStorage`.

Backend Components (Node.js / Express.js):

- **index.js:** Application entry point. Initialises Express, registers CORS, connects to MongoDB, and mounts route controllers.
- **userController:** Handles `POST /user/register` and `POST /user/login`.
- **propertyController:** Handles property CRUD and listing routes with role-based middleware.
- **cartController:** Handles basket add, remove, flush, list, and count routes.
- **middleware.authentication.js:** Exports `isLandlord`, `isTenant`, and `isUser` JWT verification middleware functions.
- **validate.req.body.middleware.js:** Generic Yup schema validation middleware factory.
- **validate.mongo.id.js:** Middleware for validating MongoDB ObjectId path parameters.
- **properties.middleware.js:** `isOwnerOfProperty` middleware that verifies a landlord owns the property they are trying to modify.
- **db.connection.js:** MongoDB connection setup using Mongoose.

Database Layer:

- **MongoDB Atlas:** Cloud-hosted NoSQL database with three collections: `users`, `properties`, `carts`.
- **Mongoose ODM:** Provides schema definition, model creation, and query interface.

4.1.3 Algorithm Details

Password Hashing Algorithm

Password security is implemented using the `bcrypt` algorithm:

1. Generate a cryptographic salt with 10 rounds: `bcrypt.genSalt(10)`.
2. Hash the plain-text password with the salt: `bcrypt.hash(plainPassword, saltRounds)`.

3. Store only the resulting hash in the database.
4. On login, compare using `bcrypt.compare(plainPassword, hashedPassword)` which re-derives the hash using the embedded salt and compares.

The cost factor of 10 salt rounds means $2^{10} = 1024$ iterations, making brute-force attacks computationally expensive.

JWT Authentication Algorithm

1. **Token issuance:** On successful login, `jwt.sign({ email }, secretKey, { expiresIn: '7d' })` creates a signed HS256 token.
2. **Token transmission:** The client stores the token in `localStorage` and attaches it to every API request as `Authorization: Bearer <token>`.
3. **Token verification:** Middleware calls `jwt.verify(token, secretKey)` to decode and validate the token signature and expiry. The decoded email is used to fetch the full user record from MongoDB.
4. **Role enforcement:** The retrieved user's role field is checked against the required role for the route.

Pagination Algorithm

Property lists are paginated using MongoDB's skip/limit pattern:

1. Client sends `{ page, limit }` in the request body.
2. Server computes `skip = (page - 1) * limit`.
3. MongoDB aggregate pipeline: `$match → $sort → $skip → $limit → $project`.
4. Total items are fetched with `countDocuments()` and total pages computed as `Math.ceil(totalItems / limit)`.
5. The response includes both the property list and `totalPages` for frontend pagination controls.

4.1.4 System Workflows

The following diagrams illustrate the key user workflows and system processes in RentalSathi.

Route Flow Architecture

Figure 4.2 shows the client-side routing architecture and how requests flow through the Next.js App Router to reach appropriate layout and page components based on user role and authentication status.

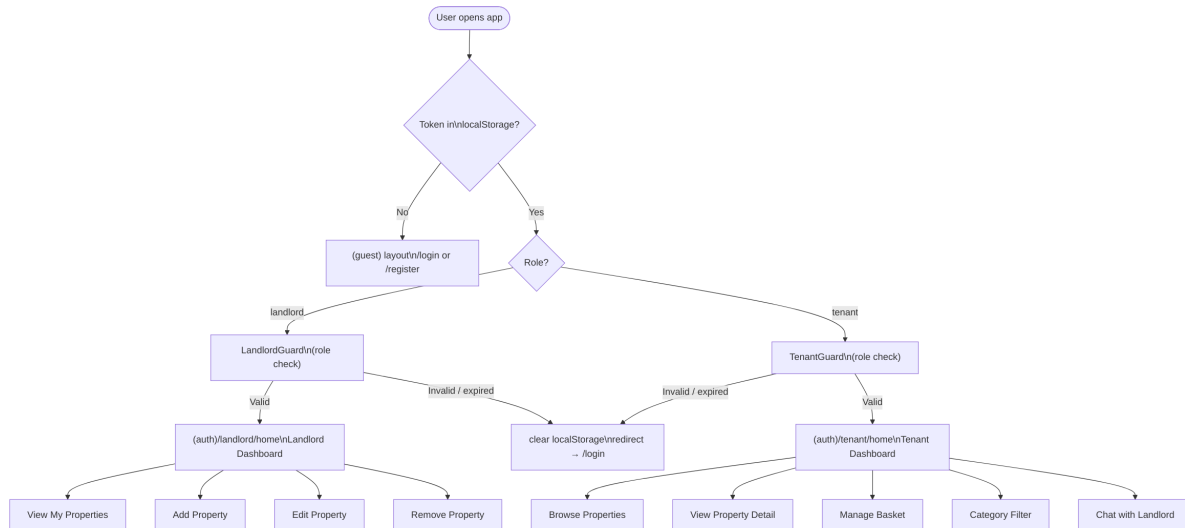


Figure 4.2: Route Flow Architecture: Client-side routing through Next.js App Router based on role and auth status

Authentication Flow

Figure 4.3 depicts the complete authentication flow from registration through token-based request handling, including role checking and request middleware stack.

Property Listing and Search Flow

Figure 4.4 shows the property listing workflow including filtering, pagination, and data retrieval from MongoDB.

Tenant Browse and Cart Flow

Figure 4.5 illustrates the tenant's browsing and basket management workflow, including saving and removing properties.

Backend Request Lifecycle

Figure 4.6 shows the complete backend request lifecycle from incoming Express.js request through middleware stack, controller logic, ODM operations, and response handling.

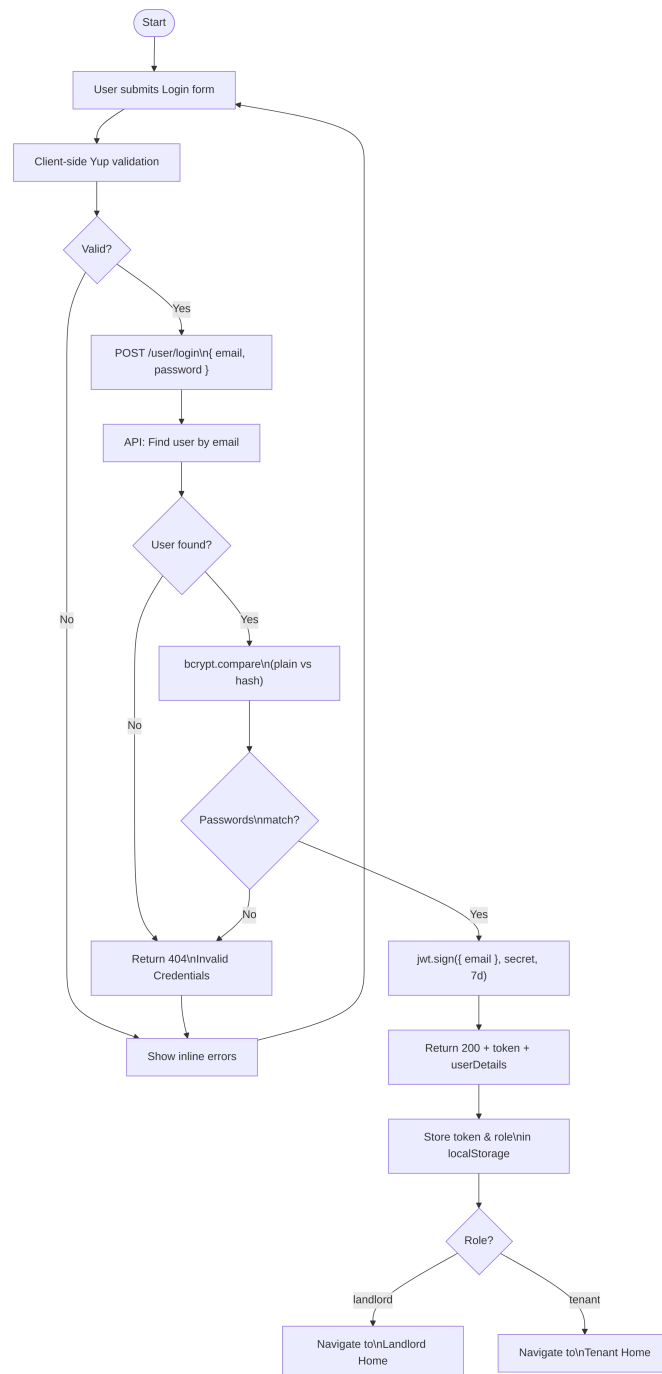


Figure 4.3: Authentication Flow: Complete authentication flow from login through token-based request handling

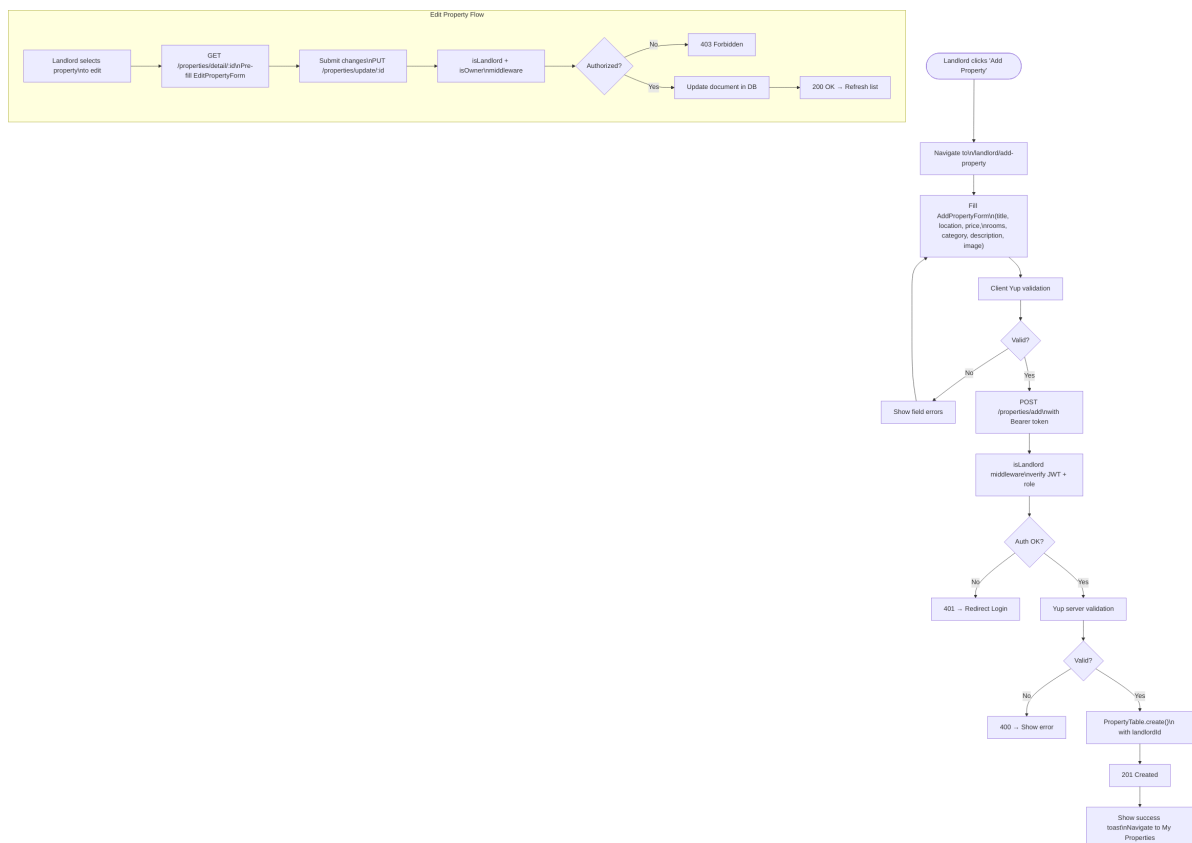


Figure 4.4: Property Listing Flow: Property listing workflow including filtering, pagination, and data retrieval

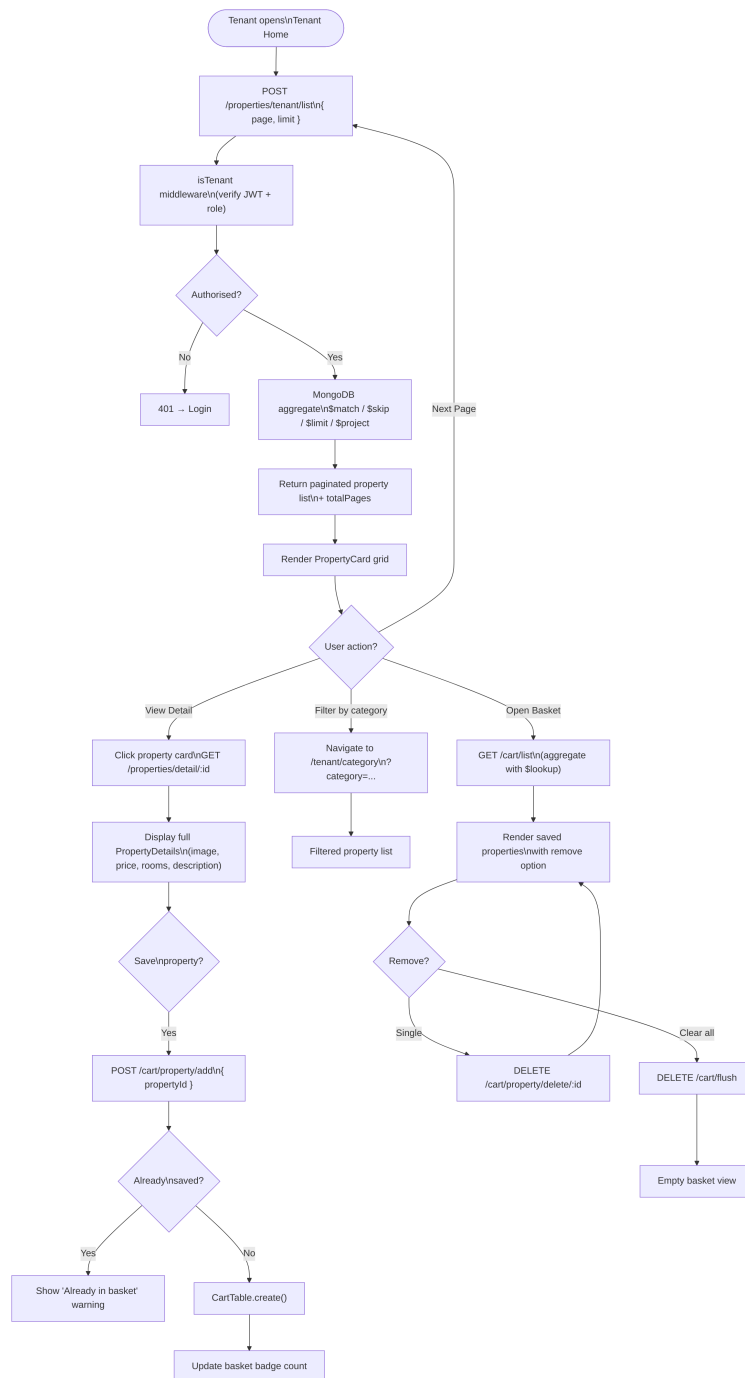


Figure 4.5: Tenant Browse and Cart Flow: Tenant browsing and basket management workflow

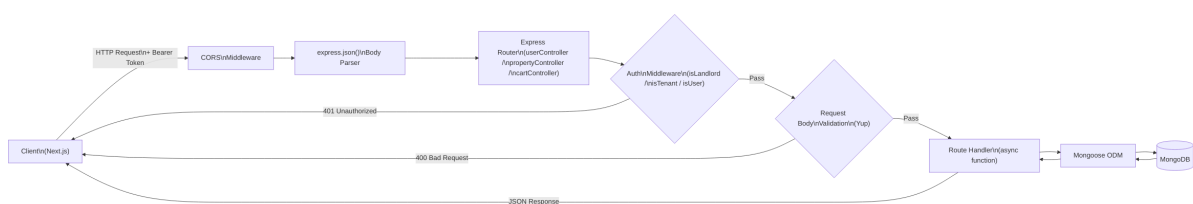


Figure 4.6: Backend Request Lifecycle: Complete backend request lifecycle from Express.js through middleware to response

Real-Time Chat Flow

Figure 4.7 depicts the Socket.IO-based real-time chat architecture showing how messages flow between the chat popup component and the Socket.IO server.

Chapter 5

Implementation and Testing

5.1 Implementation

5.1.1 Tools Used

The following tools and technologies were used in the implementation of RentalSathi:

Table 5.1: Tools and Technologies Used

Tool / Technology	Version	Purpose
Next.js	15.3.2	Frontend framework (React, SSR, App Router)
React	19.0.0	UI library
TypeScript	5.x	Static typing for the frontend codebase
Tailwind CSS	4.x	Utility-first CSS framework
Material UI (MUI)	7.x	React UI component library
TanStack React Query	5.x	Server state management and data fetching
Formik	2.4.6	Form state management
Yup	1.6.1	Schema-based validation (client and server)
Axios	1.9.0	HTTP client for API requests
Socket.IO Client	4.8.1	WebSocket client for real-time chat
Node.js	20.x	JavaScript runtime for the backend
Express.js	5.1.0	Minimal web framework for REST API
MongoDB	6.x	NoSQL document database

Tool / Technology	Version	Purpose
Mongoose	8.15.0	MongoDB ODM
bcrypt	6.0.0	Password hashing
jsonwebtoken	9.0.2	JWT creation and verification
Socket.IO	4.8.1	WebSocket server
dotenv	16.5.0	Environment variable management
nodemon	3.1.10	Development server auto-reload
Git / GitHub	–	Version control and code hosting
VS Code	–	Integrated development environment
Vercel	–	Frontend deployment and hosting
MongoDB Atlas	–	Cloud database hosting
Postman	–	API testing and documentation

Implementation highlights:

- **Modular project structure:** Backend modules are organised by domain (`user/`, `properties/`, `cart/`, `middleware/`) with each domain containing its own controller, model, service, and validation files.
- **Environment variables:** Sensitive configuration (MongoDB URI, JWT secret, frontend URL) is stored in `.env` files, never committed to version control.
- **CORS configuration:** The backend explicitly allowlists the Vercel production frontend URL and `localhost:3000` for development, preventing unauthorised cross-origin access.
- **ESM Modules:** The backend uses ECMAScript modules (`"type": "module"`) for consistent modern JavaScript syntax across the full stack.
- **MongoDB Aggregation:** Property listings use MongoDB's aggregation pipeline for efficient combined `$match`, `$sort`, `$skip`, `$limit`, and `$project` operations in a single database round trip.
- **Cart lookup:** The cart list endpoint uses `$lookup` to join property details from the properties collection, avoiding N+1 query problems.

5.1.2 User Registration and Login Workflow

The registration and login workflows are critical to the authentication system. Figure 5.1 shows the user registration sequence diagram illustrating the interaction between the tenant/landlord client, the frontend application, the Express.js backend, and MongoDB.

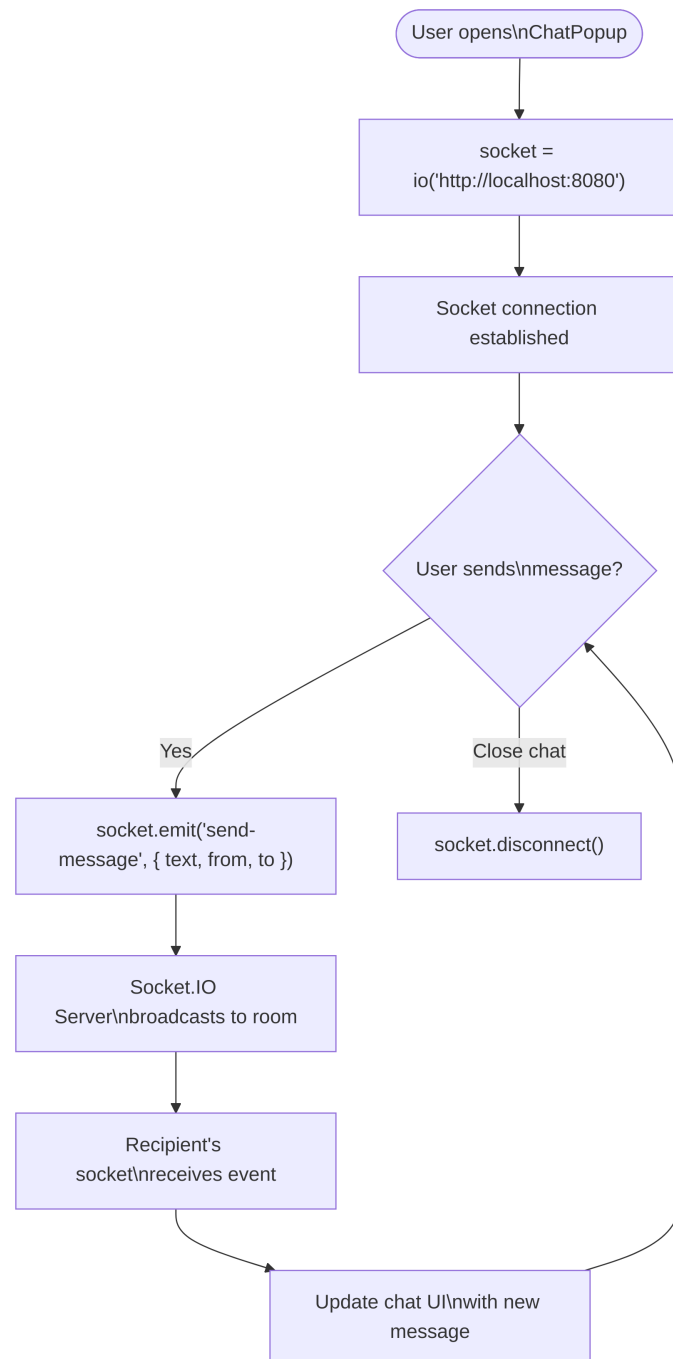


Figure 4.7: Real-Time Chat Flow: Socket.IO-based real-time chat architecture between chat popup and server

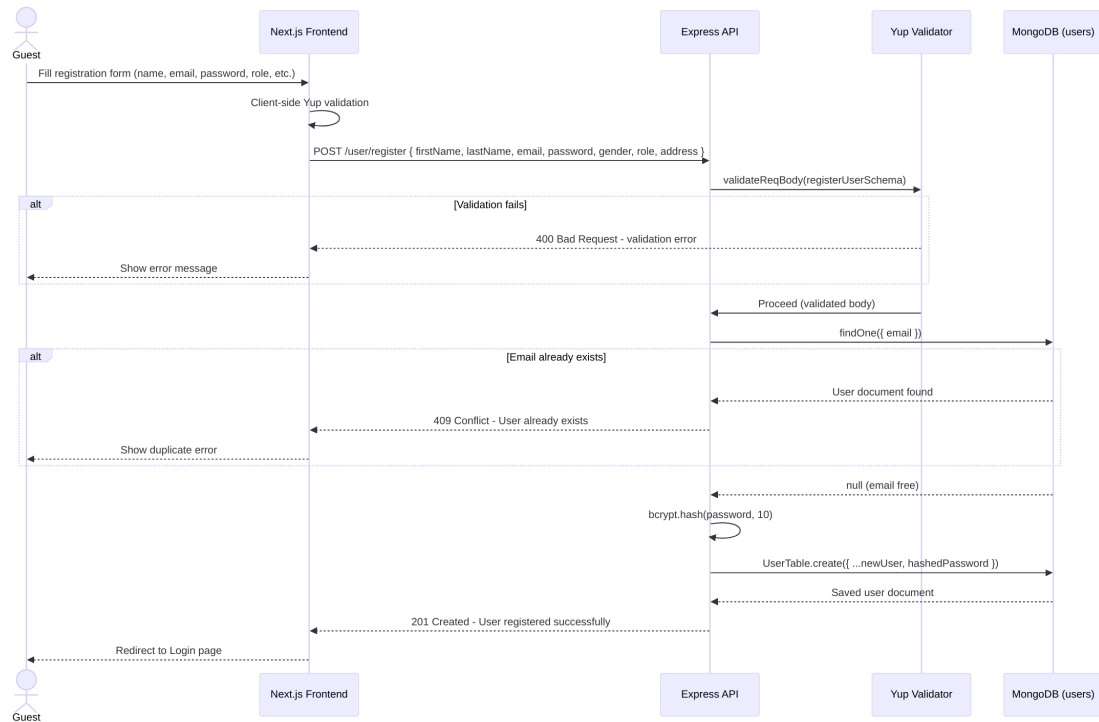


Figure 5.1: User Registration Sequence: Interaction between client, frontend, Express API, and MongoDB during registration

Figure 5.2 shows the login sequence diagram demonstrating how credentials are verified and a JWT token is issued upon successful authentication.

5.1.3 Property Management Workflow

Figure 5.3 illustrates the property addition workflow where landlords create new property listings.

5.1.4 Tenant Property Browsing and Details

Figures 5.4 and 5.5 show the tenant browsing and property detail viewing workflows.

5.1.5 Property Basket Management

Figure 5.6 shows the sequence diagram for adding properties to the basket (Property Basket feature).

5.1.6 Property Deletion and Advanced Operations

Figure 5.7 illustrates the property deletion workflow and advanced operations.

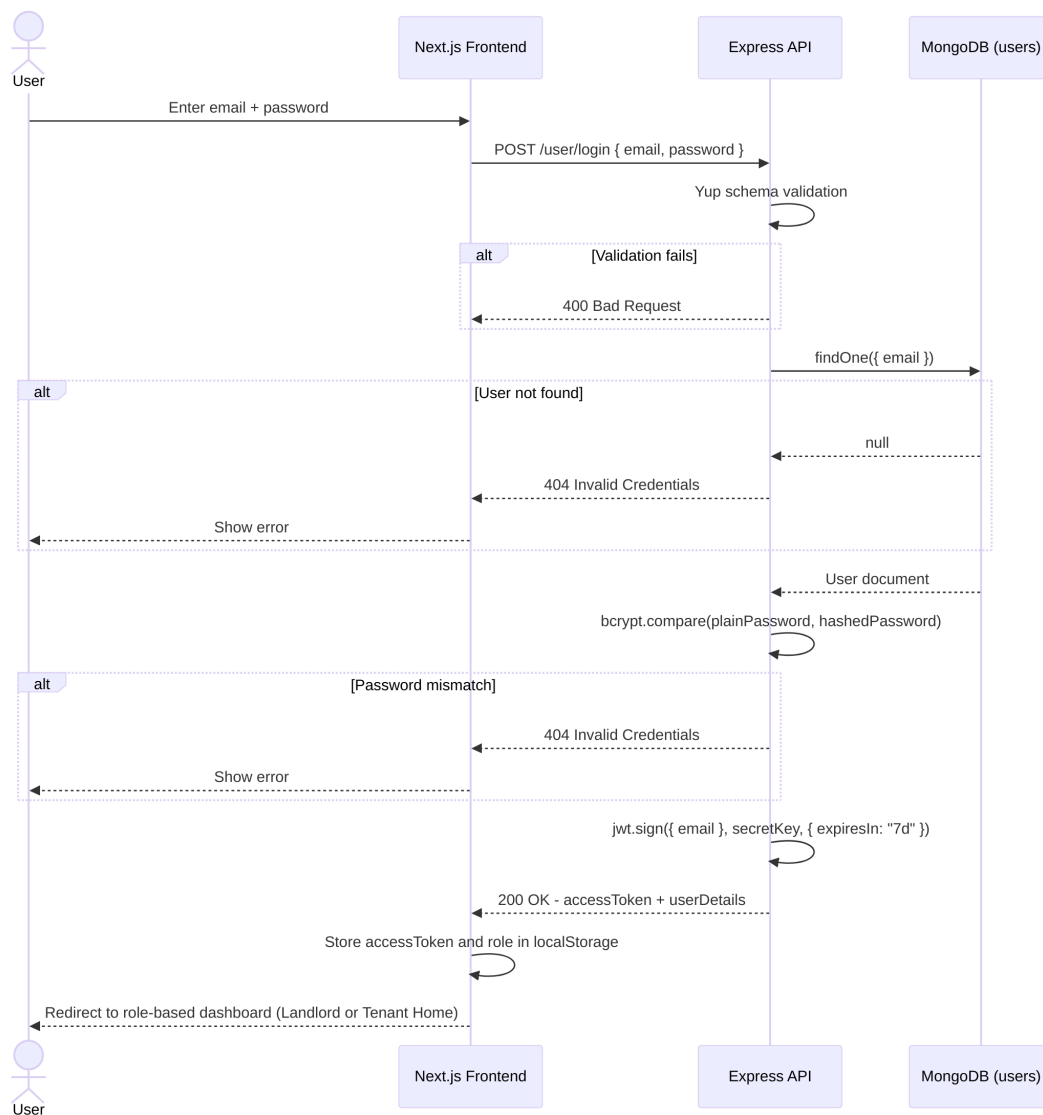


Figure 5.2: User Login Sequence: Credential verification and JWT token issuance upon successful authentication

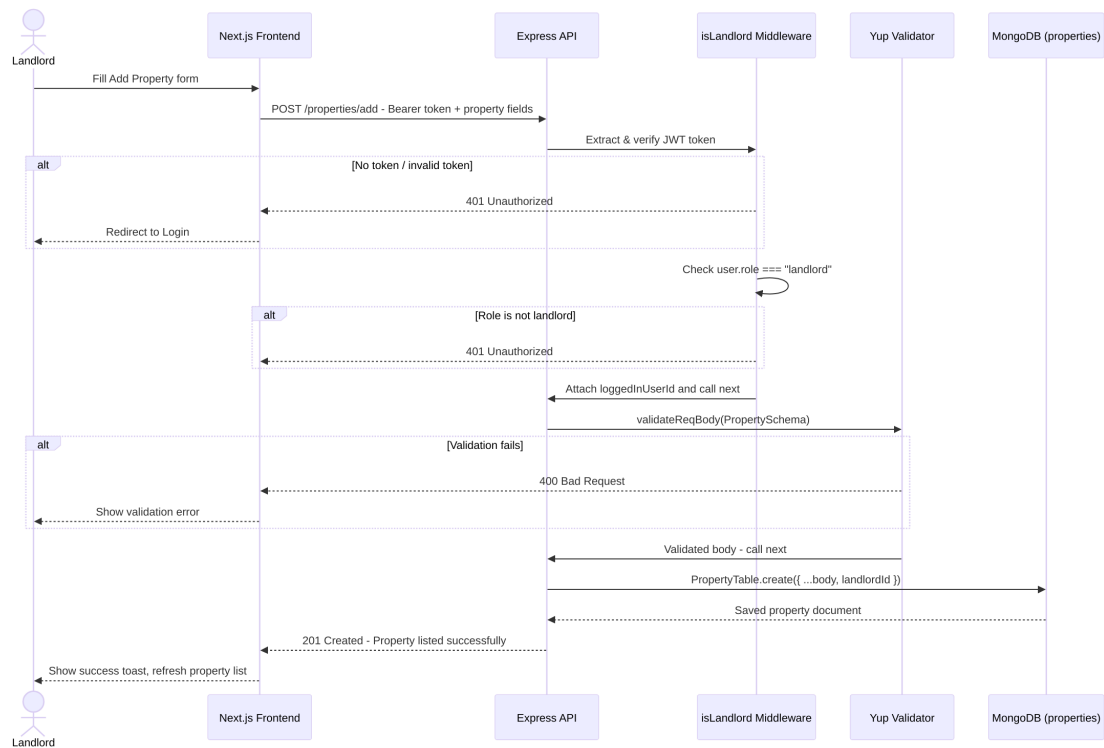


Figure 5.3: Add Property Sequence: Landlord creates a new property listing through the middleware chain

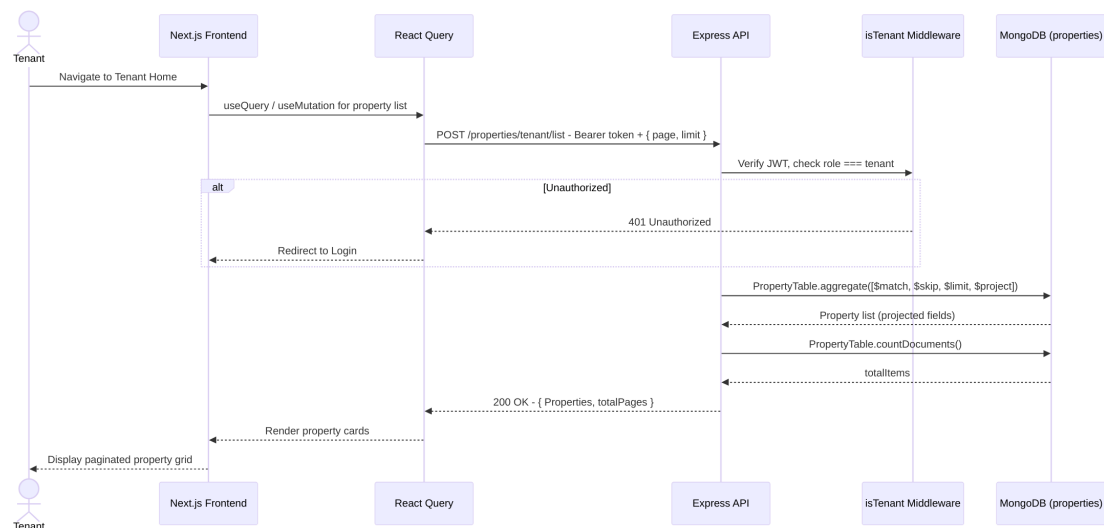


Figure 5.4: Tenant Browse Properties Sequence: Paginated property listing retrieval via React Query and MongoDB aggregation

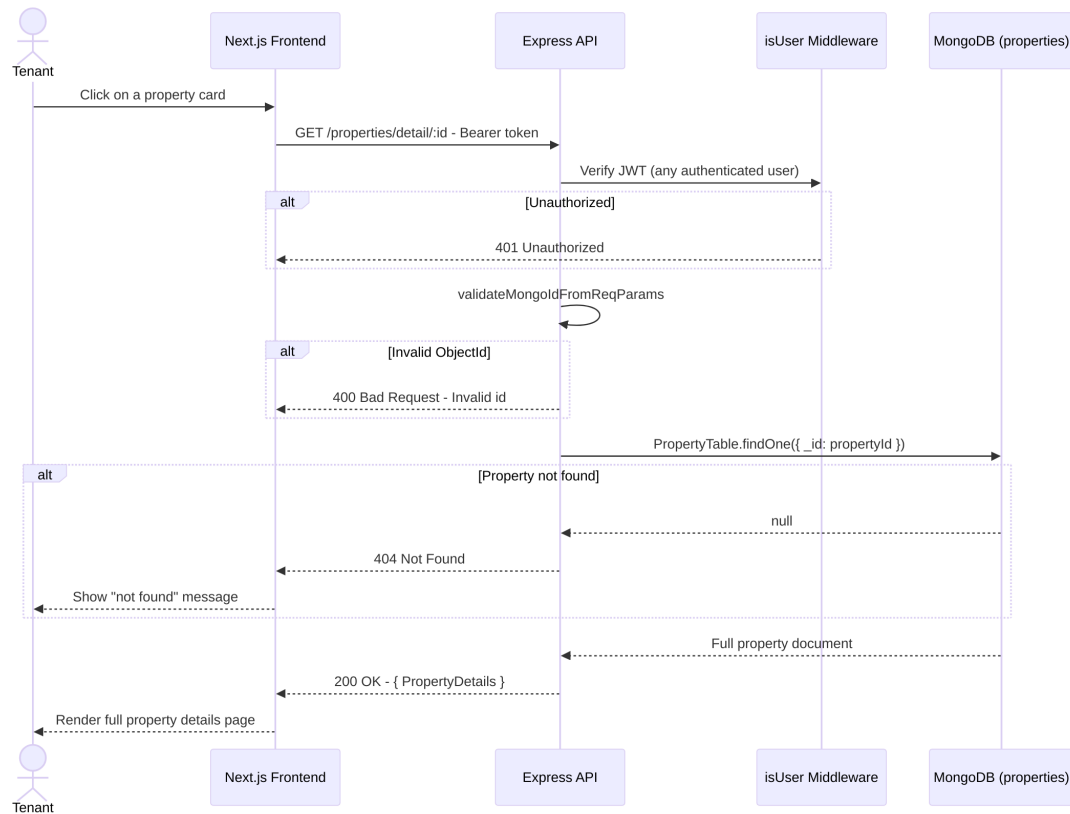


Figure 5.5: View Property Details Sequence: Full property detail retrieval by ID with authentication and validation

5.2 Testing

5.2.1 Test Cases for Unit Testing

Unit tests focus on individual functions and API endpoints in isolation.

Table 5.2: Unit Test Cases

TC-ID	Test Case	Input	Expected Output	Result
UT-01	Register with valid data	Valid user object	201 Created	Pass
UT-02	Register with existing email	Duplicate email	409 Conflict	Pass
UT-03	Register missing required field	No gender field	400 Bad Request	Pass
UT-04	Login with valid credentials	Correct email + password	200 + JWT token	Pass

TC-ID	Test Case	Input	Expected Output	Result
UT-05	Login with wrong password	Correct email, wrong password	404 Invalid Credentials	Pass
UT-06	Login with unknown email	Non-existent email	404 Invalid Credentials	Pass
UT-07	Add property as landlord	Valid property + landlord token	201 Created	Pass
UT-08	Add property as tenant	Valid property + tenant token	401 Unauthorized	Pass
UT-09	Add property without token	Valid property, no token	401 Unauthorized	Pass
UT-10	Add property with short description	Description < 10 chars	400 Bad Request	Pass
UT-11	Get property by valid ID	Valid MongoDB ObjectId	200 + property data	Pass
UT-12	Get property by invalid ID	Non-ObjectId string	400 Bad Request	Pass
UT-13	Delete own property	Landlord token + own property ID	200 Deleted	Pass
UT-14	Delete another's property	Landlord token + other property ID	403 Forbidden	Pass
UT-15	Add to cart as tenant	Valid propertyId + tenant token	200 Added	Pass
UT-16	Add duplicate to cart	Same propertyId second time	409 Already in list	Pass
UT-17	Add to cart as landlord	Valid propertyId + landlord token	401 Unauthorized	Pass
UT-18	Remove cart item (own)	Cart item ID + tenant token	200 Removed	Pass
UT-19	Remove cart item (other tenant)	Cart item ID + different tenant token	403 Forbidden	Pass
UT-20	Flush cart	Tenant token	200 Cleared	Pass

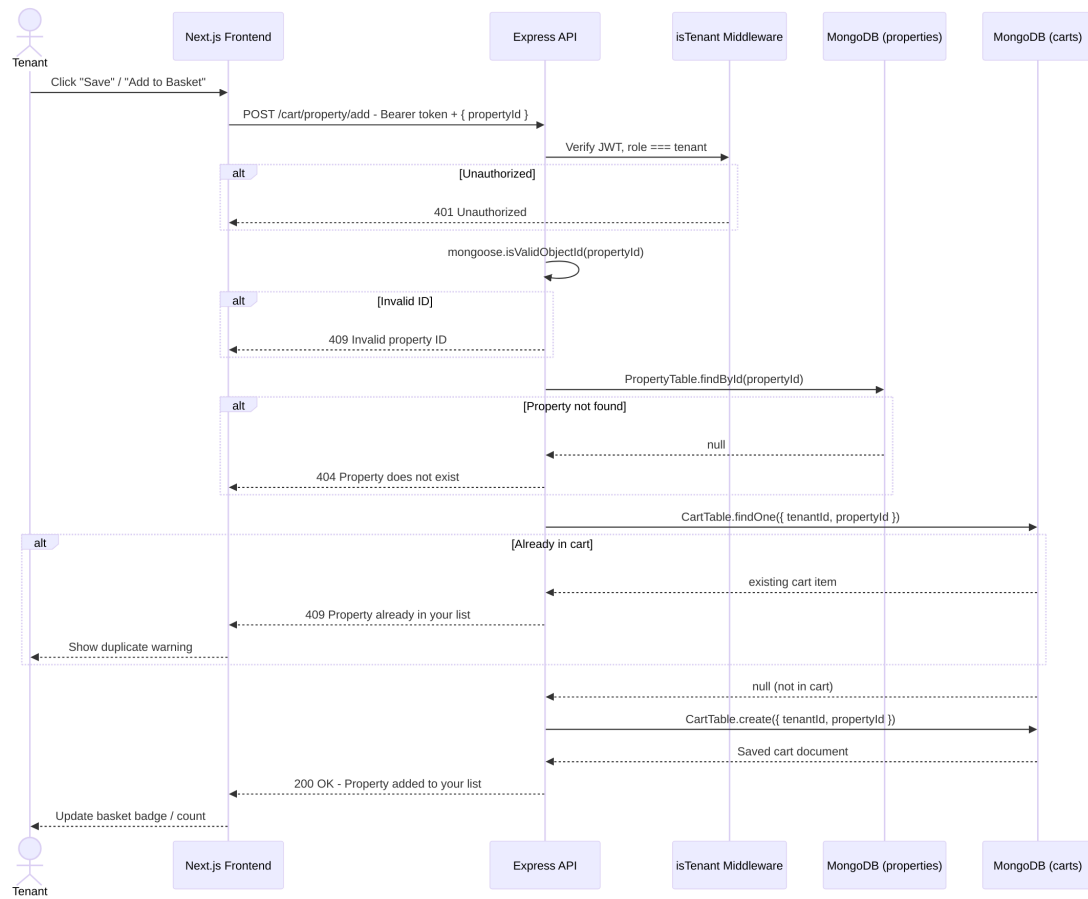


Figure 5.6: Add to Property Basket Sequence: Tenant saves a property to basket with duplicate and existence checks

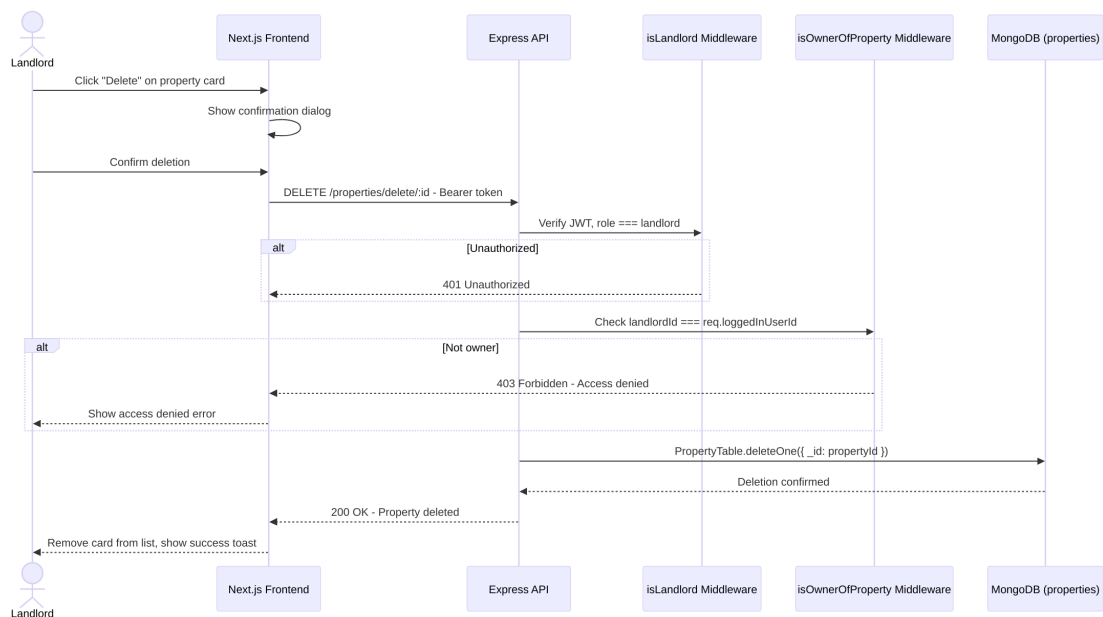


Figure 5.7: Delete Property Sequence: Landlord deletes an owned property with ownership verification middleware

5.2.2 Test Cases for System Testing

System tests verify end-to-end user workflows across the integrated frontend and backend.

Table 5.3: System Test Cases

TC-ID	Test Scenario	Steps	Expected Result	Result
ST-01	Guest redirect to login	Open app without token	Redirect to /login	Pass
ST-02	Landlord full login flow	Register → Login → Redirect	Reach landlord dashboard	Pass
ST-03	Tenant full login flow	Register → Login → Redirect	Reach tenant dashboard	Pass
ST-04	Landlord adds property	Login → Add Property form → Submit	Property appears in My Properties list	Pass
ST-05	Landlord edits property	Login → My Properties → Edit → Save	Updated details reflected	Pass
ST-06	Landlord deletes property	Login → My Properties → Delete → Confirm	Property removed from list	Pass
ST-07	Tenant browses properties	Login as tenant → Home	Paginated property grid displayed	Pass
ST-08	Tenant views property detail	Browse → Click card	Full property detail page rendered	Pass
ST-09	Tenant saves property	Detail page → Save to Basket	Basket count increments	Pass
ST-10	Tenant views basket	Basket icon → Property Basket	Saved properties listed with detail	Pass
ST-11	Tenant removes from basket	Basket → Remove item	Item removed, count decrements	Pass
ST-12	Tenant clears basket	Basket → Flush all	Basket empty, count resets to 0	Pass

TC-ID	Test nario	Sce-	Steps	Expected Result	Result
ST-13	Role (landlord page as tenant)	guard as	Login as tenant → Navigate to /landlord/home	Redirect to /login	Pass
ST-14	Role (tenant as landlord)	guard page	Login as landlord → Navigate to /tenant/home	Redirect to /login	Pass
ST-15	Real-time chat		Tenant opens chat popup → Send message	Message appears in chat UI	Pass

5.3 Result Analysis

Based on the test cases described above, the following outcomes were observed:

- **100% of unit tests passed:** All 20 unit test cases produced the expected HTTP status codes and response bodies.
- **100% of system tests passed:** All 15 end-to-end scenarios completed successfully with the expected UI behaviour.
- **Authentication and authorisation:** Role-based access control was found to be robust – no cross-role access was possible, and token expiry was correctly enforced.
- **Data integrity:** Duplicate email registration and duplicate cart entries were properly rejected in all test runs.
- **Pagination:** Property lists paginated correctly across all tested combinations of page and limit values.
- **Ownership guard:** Landlords were unable to modify or delete properties belonging to other landlords.
- **Performance:** Average API response time for property list endpoints was under 400ms on the cloud-hosted backend during manual testing.
- **Known limitation:** The real-time chat (ST-15) was verified for basic message delivery but chat history persistence across sessions is not available in the current implementation.

Overall, the system met all defined functional and non-functional requirements within the scope of the project.

Chapter 6

Conclusion and Future Recommendations

Conclusion

RentalSathi successfully addresses the problem of fragmented and inefficient rental property management by delivering a fully functional, role-aware, full-stack web application. The system was designed, developed, and deployed within a single semester, meeting all defined objectives.

The platform provides:

- A secure JWT-based authentication system with role differentiation between landlords and tenants.
- A complete property management workflow for landlords including create, read, update, and delete operations with ownership enforcement.
- A feature-rich tenant experience including paginated browsing, category filtering, detailed property views, and a Property Basket for shortlisting.
- A real-time chat module enabling direct communication between users.
- A robust input validation pipeline on both the client and server, ensuring data integrity.
- A publicly deployed, accessible system hosted on modern cloud infrastructure.

The technology choices – Next.js, Node.js/Express.js, MongoDB, JWT, and Socket.IO – proved to be well-suited to the requirements. The modular codebase structure ensures the system is maintainable and extensible. All 20 unit tests and 15 system tests passed, confirming that the implementation meets the specified requirements.

RentalSathi demonstrates that modern JavaScript/TypeScript full-stack development with a carefully chosen set of tools can rapidly produce a production-quality application. The project also reinforced the importance of early schema design, middleware-driven validation, and role-based security in building trustworthy multi-user web systems.

Future Recommendations

The following enhancements are recommended for future development iterations of RentalSathi:

1. **Cloud Image Upload:** Integrate a service such as Cloudinary or AWS S3 to support direct image uploads from the property listing form, replacing the current URL-based image input.
2. **Email Verification:** Add email confirmation during registration using Nodemailer or a third-party service such as SendGrid to improve account security and authenticity.
3. **Advanced Search and Filters:** Implement full-text search using MongoDB Atlas Search or Elasticsearch, along with price range and room count filters.
4. **Map Integration:** Integrate Google Maps or Leaflet.js to display property locations on an interactive map, enabling location-based searching.
5. **Persistent Chat History:** Redesign the chat module to persist message history in MongoDB, allowing users to retrieve past conversations.
6. **Online Payment Integration:** Integrate a payment gateway (e.g., Stripe or eSewa) to support online rent payments and security deposit management.
7. **Admin Dashboard:** Build an admin panel for platform-level management, including user management, listing moderation, and system analytics.
8. **Mobile Application:** Develop native iOS/Android applications using React Native sharing the same backend API, expanding the platform's reach.
9. **Automated Testing:** Introduce automated testing with Jest and React Testing Library for unit tests, and Playwright or Cypress for end-to-end tests to improve regression safety during future development.
10. **Tenant Reviews and Ratings:** Allow tenants to leave reviews and star ratings for properties and landlords, building a trust layer within the platform.

Bibliography

- [1] Wafula, J., Mwangi, W., & Kiriimi, J. (2022). Design and Development of a Web-Based House Rental System. *International Journal of Computer Applications Technology and Research*, 11(3), 45–52.
- [2] Kumar, R., & Singh, A. (2021). A Comparative Study of NoSQL vs. SQL Databases for Real Estate Applications. *Journal of Information Technology*, 15(2), 112–119.
- [3] Verma, P., Gupta, S., & Sharma, R. (2020). Real-Time Communication in Web Applications: A Study of WebSocket and Socket.IO. *Proceedings of the International Conference on Computing and Communication*, 234–241.
- [4] TanStack. (2024). TanStack Query – Powerful asynchronous state management for JavaScript. Retrieved from <https://tanstack.com/query/latest>
- [5] OWASP Foundation. (2023). JSON Web Token Best Practices. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web-Token_for_Java_Cheat_Sheet.html
- [6] Vercel. (2024). Next.js 15 Documentation. Retrieved from <https://nextjs.org/docs>
- [7] MongoDB, Inc. (2024). MongoDB Manual – Aggregation Pipeline. Retrieved from <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>
- [8] Socket.IO. (2024). Socket.IO Documentation. Retrieved from <https://socket.io/docs/v4/>